

Declaring non-type template arguments with auto

Document number: P0127R1
Date: 2016-03-04
Project: Programming Language C++, Evolution Working Group
Revises: [P0127R0](#)
Reply-to: James Touton <bekenn@gmail.com>
Mike Spertus, Symantec <mike_spertus@symantec.com>

Table of Contents

1. [Table of Contents](#)
2. [What's New](#)
3. [Introduction](#)
4. [Motivation and Scope](#)
5. [Impact On the Standard](#)
6. [Design Decisions](#)
7. [Technical Specifications](#)
8. [Wording](#)
9. [Acknowledgments](#)

What's New

Changes since P0127R0

- Added wording for a hypothetical post-Concepts draft of the Standard.

Introduction

This paper proposes allowing non-type template parameters to be declared with the `auto` placeholder type specifier. The desired effect is that the type of the corresponding non-type template argument be automatically deduced, much like similar syntax works for polymorphic lambdas. This more focused proposal follows EWG's recommendations from the Spring 2015 Lenexa meeting.

The existing workaround is that the type of a non-type template parameter must be explicitly specified, which leads to unnecessary verbosity and reduced flexibility when writing a template intended to take constant arguments of any type. Example:

```
template <typename T, T v> struct S { };    // definition
S<decltype(x), x> s;                       // instantiation
```

The example makes use of `decltype` to retrieve the type of `x` (a compile-time constant) before passing both the type and the value of `x` to `s`. The goal is to be able to modify the declaration of `s` such that the type of `x` doesn't need to be passed as a separate template argument, resulting in this simpler instantiation:

```
S<x> s; // desired instantiation
```

This can be achieved by allowing use of the `auto` keyword in template parameter lists.

```
template <auto v> struct S;                // type of v is deduced
```

Motivation and Scope

Consider a generic function call logger for an application that provides callback function pointers to a library. The logger should print the name of the function, the argument values, and the result of a call to any callback. In order to avoid calling the logger from within a callback function (and thus having to modify each function to support the logger), the logger itself is passed to the library in place of the callback function, and the logger passes the arguments along to the callback function. This implies that the logger for a callback function must match the callback function's type so that the library can call it directly.

It is desirable that the instantiation syntax for the logger be simple; the following seems perfectly reasonable:

```
// can't specify string literals as template arguments, so provide a character array instead
static constexpr char cname[] = "my_callback";
```

```
void initialize()
{
    library::register_callback(logger<my_callback, cbname>);
}
```

In order for this to work, `logger` must be a template that takes a function pointer and a character pointer as arguments. If the type of the function is fixed, this is no problem:

```
// log any function with the signature int(int)
template <int (* f)(int), const char* name>
int logger(int arg)
{
    cout << name << '(' << arg << ')';
    int result = f(arg);
    cout << " -> " << result << endl;
    return result;
}
```

If the type of the function is not fixed, things get more complicated:

```
// log each argument in a comma-separated list
template <class... Args> void log_args(Args... args);

// struct template that accepts a function pointer and a name
template <class F, F f, const char* name> struct fn_logger;

// use partial specialization to constrain f
// note that a second specialization would be needed to support functions returning void
template <class R, class... Args, R (* f)(Args...), const char* name>
struct fn_logger<R (*)(Args...), f, name>
{
    // call f, logging arguments and result
    static R call(Args... args)
    {
        cout << name << '(';
        log_args(args...);
        cout << ')';
        auto result = f(args...);
        cout << " -> " << result << endl;
        return result;
    }
};

// variable template to simplify use of fn_logger
template <class F, F f, const char* name> constexpr auto logger = fn_logger<F, f, name>::call;
```

The instantiation syntax also gets more complicated, because the type of the function must be passed as an additional argument:

```
// can't specify string literals as template arguments, so provide a character array instead
static constexpr char cbname[] = "my_callback";
void initialize()
{
    library::register_callback(decltype(&my_callback), logger<my_callback, cbname>);
}
```

Using auto

The template parameter list syntax can be extended in a simple and natural way using the `auto` keyword to indicate that the type of a value parameter is deduced at the point of instantiation:

```
template <auto x> constexpr auto constant = x;

auto v1 = constant<5>;           // v1 == 5, decltype(v1) is int
auto v2 = constant<true>;        // v2 == true, decltype(v2) is bool
auto v3 = constant<'a'>;         // v3 == 'a', decltype(v3) is char
```

The usual type modifiers may be used to constrain the type of the value parameter without the use of partial specialization:

```
// p must be a pointer to const something
template <const auto* p> struct S;
```

Partial specialization may be used to switch on the type of a value parameter:

```
template <auto x> struct S;
template <int n>
struct S<n>
{

```

```
const char* type_name = "int";
};
```

Here is what the logger would look like using auto:

```
// log each argument in a comma-separated list
template <class... Args> void log_args(Args... args);

// struct template that accepts a function pointer and a name
template <auto f, const char* name> struct fn_logger;

// use partial specialization to constrain f
// note that a second specialization would be needed to support functions returning void
template <class R, class... Args, R (* f)(Args...), const char* name>
struct fn_logger<f, name>
{
    // call f, logging arguments and result
    static R call(Args... args)
    {
        cout << name << '(';
        log_args(args...);
        cout << ')';
        auto result = f(args...);
        cout << " -> " << result << endl;
        return result;
    }
};

// variable template to simplify use of fn_logger
template <auto f, const char* name> constexpr auto logger = fn_logger<f, name>::call;
```

The function type no longer needs to be explicitly specified, which means the instantiation can go back to the desired form:

```
library::register_callback(logger<my_callback, cbname>);
```

As variadic template parameters

When auto appears as the type specifier for a parameter pack, it signifies that the type for each corresponding argument should be independently deduced:

```
// List of heterogeneous constant values
// same as template <auto v1, auto v2, auto v3, ...>
template <auto... vs> struct value_list { };

// Retrieve the nth value in a list of values
template <size_t n, auto... vs> struct nth_value;
template <size_t n, auto v1, auto... vs>
struct nth_value<n, v1, vs...>
{
    static constexpr auto value = nth_value<n - 1, vs...>::value;
};
template <auto v1, auto... vs>
struct nth_value<0, v1, vs...>
{
    static constexpr auto value = v1;
};
```

A list of homogeneous constant values can be constructed with the aid of decltype:

```
// List of homogeneous constant values
template <auto v1, decltype(v1)... vs> struct typed_value_list { };
```

Impact On the Standard

The proposed feature adds no keywords and does not change the meaning of any existing code.

Opportunity cost

There is an opportunity cost associated with adopting this particular meaning for the auto keyword in this context. It has been suggested that auto could be used to allow for template parameters accepting *any kind* of template argument, be it a type, a value, a template, or any other construct that templates may accept at any point in the future.

Such a feature is desirable, but the use of the auto keyword for it is not. There is no existing context in which auto acts as anything other than a stand-in for a type name; consistency with the rest of the language dictates that auto behave as spelled out

in this paper.

Compatibility with Concepts

This proposal is intended to be fully compatible with the Concepts TS. Because Concepts introduces new rules for dealing with placeholders, care had to be taken to avoid impacting any features of Concepts. The wording provided in this proposal is careful to specify that the rules introduced here are only applicable to placeholders *designated by `auto` or `decltype(auto)`*. Concepts does not establish any new behaviors for `auto` that conflict with this proposal.

Design Decisions

A few people have suggested that all values in a parameter pack introduced by `auto` should have the same type. The rationale seems to be that because `auto` can be replaced by a single type name in a multiple variable definition, the same should be true here:

```
auto x = 3.5, y = "hello"; // error, x and y must have the same type
```

This approach is comparatively inflexible, in that it does not allow variadic lists of heterogeneous values. Additionally, the behavior specified in this document mirrors the existing behavior of the `typename` and `class` keywords in this context:

```
// same as template <typename T1, typename T2, typename T3, ...>
template <typename... Ts> struct type_list { };
```

```
// same as template <auto v1, auto v2, auto v3, ...>
template <auto... vs> struct value_list { };
```

Technical Specifications

- The `auto` keyword, when it appears in a template parameter list, signifies that the associated template parameter is a value parameter and that the type of the value parameter is to be deduced at the point of template instantiation.
- The `auto` keyword, when it introduces a template parameter pack, signifies that each element in the parameter pack is a value parameter and that the types of the value parameters are to be independently deduced at the point of template instantiation.
- Type deduction for template parameters introduced with the `auto` keyword follows the same rules as specified for function template argument type deduction.

Wording

Relative to the Working Draft

All modifications are presented relative to N4567.

Modify §7.1.6.4 [dcl.spec.auto] paragraph 5:

A placeholder type can also be used in declaring a variable in the *condition* of a selection statement (6.4) or an iteration statement (6.5), in the *type-specifier-seq* in the *new-type-id* or *type-id* of a *new-expression* (5.3.4), in a *for-range-declaration*, and in declaring a static data member with a brace-or-equal-initializer that appears within the member-specification of a class definition (9.4.2), and in the *decl-specifier-seq* in the *parameter-declaration* of a *template-parameter* (14.1).

Modify §7.1.6.4 [dcl.spec.auto] paragraph 7:

When a variable *or non-type template parameter* declared using a placeholder type is initialized, or a return statement occurs in a function declared with a return type that contains a placeholder type, the deduced return type, *or variable type, or template parameter type* is determined from the type of its initializer. In the case of a return with no operand or with an operand of type `void`, the declared return type shall be `auto` and the deduced return type is `void`. Otherwise, let τ be the declared type of the variable, *template parameter*, or return type of the function. If the placeholder is the *auto type-specifier*, the deduced type is determined using the rules for template argument deduction. If the initialization is direct-list-initialization then the *braced-init-list* shall contain only a single *assignment-expression* ι . If the deduction is for a return statement and the initializer is a *braced-init-list* (8.5.4), the program is ill-formed. Otherwise, obtain ρ from τ by replacing the occurrences of `auto` with either a new invented type template parameter υ or, if the initialization is copy-list-initialization, with `std::initializer_list< $\upsilon$ >`.

Deduce a value for `u` using the rules of template argument deduction from a function call (14.8.2.1), where `P` is a function template parameter type and the corresponding argument is the initializer, or `L` in the case of direct-list-initialization. If the deduction fails, the declaration is ill-formed. Otherwise, the type deduced for the variable or return type is obtained by substituting the deduced `u` into `P`. [*Example:*

```
auto x1 = { 1, 2 };           // decltype(x1) is std::initializer_list<int>
auto x2 = { 1, 2.0 };        // error: cannot deduce element type
auto x3{ 1, 2 };             // error: not a single element
auto x4 = { 3 };             // decltype(x4) is std::initializer_list<int>
auto x5{ 3 };                // decltype(x5) is int
```

—end example]

[*Example:*

```
const auto &i = expr;
```

The type of `i` is the deduced type of the parameter `u` in the call `f(expr)` of the following invented function template:

```
template <class U> void f(const U& u);
```

—end example]

If the placeholder is the `decltype(auto)` *type-specifier*, the declared type of the variable or return type of the function shall be the placeholder alone. The type deduced for the variable or return type is determined as described in 7.1.6.2, as though the *initializer-clause* or *expression-list* of the *initializer* or the *constant-expression of the corresponding template-argument* or the *expression* of the *return* statement had been the operand of the `decltype`. [*Example:*

```
int i;
int&& f();
auto      x2a(i);           // decltype(x2a) is int
decltype(auto) x2d(i);       // decltype(x2d) is int
auto      x3a = i;          // decltype(x3a) is int
decltype(auto) x3d = i;      // decltype(x3d) is int
auto      x4a = (i);        // decltype(x4a) is int
decltype(auto) x4d = (i);    // decltype(x4d) is int&
auto      x5a = f();        // decltype(x5a) is int
decltype(auto) x5d = f();    // decltype(x5d) is int&&
auto      x6a = { 1, 2 };   // decltype(x6a) is std::initializer_list<int>
decltype(auto) x6d = { 1, 2 }; // error, { 1, 2 } is not an expression
auto      *x7a = &i;        // decltype(x7a) is int*
decltype(auto)*x7d = &i;     // error, declared type is not plain decltype(auto)
```

—end example]

Modify §14.1 [temp.param] paragraph 4:

A non-type *template-parameter* shall have one of the following (optionally *cv-qualified*) types:

- integral or enumeration type,
- pointer to object or pointer to function,
- lvalue reference to object or lvalue reference to function,
- pointer to member,
- `std::nullptr_t`,
- a type that contains a placeholder type designated by `auto` or `decltype(auto)` (7.1.6.4).

Modify §14.3.2 [temp.arg.nontype] paragraph 1:

A *template-argument* for a non-type *template-parameter* shall be a converted constant expression (5.20) of the type of the *template-parameter*. If the type of the *template-parameter* contains a placeholder type (7.1.6.4, 14.1), the deduced parameter type is determined from the type of the *template-argument* as described in 7.6.1.4. If a deduced parameter type is not permitted for a *template-parameter* declaration (14.1), the program is ill-formed. For a non-type *template-parameter* of reference or pointer type, the value of the constant expression shall not refer to (or for a pointer type, shall not be the address of):

- a subobject (1.8),
- a temporary object (12.2),

- a string literal (2.13.5),
- the result of a typeid expression (5.2.8), or
- a predefined `__func__` variable (8.4.1).

[*Note:* If the *template-argument* represents a set of overloaded functions (or a pointer or member pointer to such), the matching function is selected from the set (13.4). —*end note*]

Modify §14.3.2 [temp.arg.nontype] paragraph 2:

[*Example:*

```
template<const int* pci> struct X { /* ... */ };
int ai[10];
X<ai> xi; // array to pointer and qualification conversions

struct Y { /* ... */ };
template<const Y& b> struct Z { /* ... */ };
Y y;
Z<y> z; // no conversion, but note extra cv-qualification

template<int (&pa)[5]> struct W { /* ... */ };
int b[5];
W<b> w; // no conversion

void f(char);
void f(int);

template<void (*pf)(int)> struct A { /* ... */ };
A<&f> a; // selects f(int)

template<auto n> struct B { /* ... */ };
B<5> b1; // OK: template parameter type is int
B<'a'> b2; // OK: template parameter type is char
B<2.5> b3; // error: template parameter type cannot be double
```

—*end example*]

Modify §14.5.5 [temp.class.spec] paragraph 8 list item 8.2:

The type of a template parameter corresponding to a specialized non-type argument shall not be dependent on a parameter of the specialization that appears as a *type-specifier* in the *type-specifier-seq* of an argument in the argument list of the specialization. Otherwise, the corresponding arguments are deduced from the type of the non-type argument. [*Example:*

```
template <class T, T t> struct C {};
template <class T> struct C<T, 1>; // error
template< int X, int (*array_ptr)[X] > class A {};
int array[5];
template< int X > class A<X,&array> { }; // error
template <auto f> class B {};
template <class R, class... Args, R (* f)(Args...)> class B<f> {} // ok
```

—*end example*]

Modify §14.6.2.2 [temp.dep.expr] paragraph 3:

An *id-expression* is type-dependent if it contains

- an *identifier* associated by name lookup with one or more declarations declared with a dependent type,
- an *identifier* associated by name lookup with a non-type *template-parameter* declared with a type that contains a placeholder type (7.1.6.4).
- an *identifier* associated by name lookup with one or more declarations of member functions of the current instantiation declared with a return type that contains a placeholder type ~~(7.1.6.4)~~,
- the *identifier* `__func__` (8.4.1), where any enclosing function is a template, a member of a class template, or a generic lambda,
- a *template-id* that is dependent,
- a *conversion-function-id* that specifies a dependent type, or

—a *nested-name-specifier* or a *qualified-id* that names a member of an unknown specialization;

or if it names a dependent member of the current instantiation that is a static data member of type “array of unknown bound of T” for some T (14.5.1.3). Expressions of the following forms are type-dependent only if the type specified by the *type-id*, *simple-type-specifier* or *new-type-id* is dependent, even if any subexpression is type-dependent:

```
simple-type-specifier ( expression-listopt )  
::opt new new-placementopt new-type-id new-initializeropt  
::opt new new-placementopt ( type-id ) new-initializeropt  
dynamic_cast < type-id > ( expression )  
static_cast < type-id > ( expression )  
const_cast < type-id > ( expression )  
reinterpret_cast < type-id > ( expression )  
( type-id ) cast-expression
```

Modify §14.8.2.5 [temp.deduct.type] paragraph 5:

The non-deduced contexts are:

- The *nested-name-specifier* of a type that was specified using a *qualified-id*.
- The *expression* of a *decltype-specifier*.
- A non-type template argument or an array bound in which a subexpression references a template parameter.
[*Note: Deduction is still performed based on the type of a non-type template argument. —end note*]
- A template parameter used in the parameter type of a function parameter that has a default argument that is being used in the call for which argument deduction is being done.
- A function parameter for which argument deduction cannot be done because the associated function argument is a function, or a set of overloaded functions (13.4), and one or more of the following apply:
 - more than one function matches the function parameter type (resulting in an ambiguous deduction), or
 - no function matches the function parameter type, or
 - the set of functions supplied as an argument contains one or more function templates.
- A function parameter for which the associated argument is an initializer list (8.5.4) but the parameter does not have a type for which deduction from an initializer list is specified (14.8.2.1). [*Example*:

```
template<class T> void g(T);  
g({1,2,3}); // error: no argument deduced for T
```

—*end example*]

- A function parameter pack that does not occur at the end of the *parameter-declaration-list*.

Relative to the Concepts TS

All modifications are presented relative to ISO/IEC 14882:2014 (N4141) as amended first by the Concepts TS (N4553) and then incorporating changes to the working draft up through N4567.

Add a new paragraph after §7.1.6.4 [dcl.spec.auto] paragraph 8:

A placeholder type designated by `auto` or `decltype(auto)` can also be used in the *decl-specifier-seq* in the *parameter-declaration* of a *template-parameter* (14.1).

Modify §7.1.6.4 [dcl.spec.auto] paragraph 10 (paragraph 11 here):

When a variable or non-type template parameter declared using a placeholder type is initialized, or a return statement occurs in a function declared with a return type that contains a placeholder type, the deduced return type, ~~or~~ variable type, or template parameter type is determined from the type of its initializer. In the case of a `return` with no operand or with an operand of type `void`, the declared return type shall be `auto` and the deduced return type is `void`. Otherwise, let τ be the declared type of the variable, template parameter, or return type of the function. If the placeholder is the `auto` *type-specifier*, the deduced type is determined using the rules for template argument deduction. If the initialization is direct-list-initialization then the *braced-init-list* shall contain only a single *assignment-expression* $\mathbf{L}$. If the deduction is for a return statement and the initializer is a *braced-init-list* (8.5.4), the program is ill-formed. Otherwise, obtain $\mathbf{P}$ from τ by replacing the occurrences of `auto` with either a new

invented type template parameter `U` or, if the initialization is copy-list-initialization, with `std::initializer_list<U>`. Deduce a value for `U` using the rules of template argument deduction from a function call (14.9.2.1), where `P` is a function template parameter type and the corresponding argument is the initializer, or `L` in the case of direct-list-initialization. If the deduction fails, the declaration is ill-formed. Otherwise, the type deduced for the variable or return type is obtained by substituting the deduced `U` into `P`. [*Example*:

```
auto x1 = { 1, 2 };      // decltype(x1) is std::initializer_list<int>
auto x2 = { 1, 2.0 };   // error: cannot deduce element type
auto x3{ 1, 2 };        // error: not a single element
auto x4 = { 3 };        // decltype(x4) is std::initializer_list<int>
auto x5{ 3 };           // decltype(x5) is int
```

—end example]

[*Example*:

```
const auto &i = expr;
```

The type of `i` is the deduced type of the parameter `u` in the call `f(expr)` of the following invented function template:

```
template <class U> void f(const U& u);
```

—end example]

If the placeholder is the `decltype(auto)` *type-specifier*, the declared type of the variable or return type of the function shall be the placeholder alone. The type deduced for the variable or return type is determined as described in 7.1.6.2, as though the *initializer-clause* or *expression-list* of the *initializer* or the *constant-expression* of the *corresponding template-argument* or the *expression* of the `return` statement had been the operand of the `decltype`. [*Example*:

```
int i;
int&& f();
auto      x2a(i);      // decltype(x2a) is int
decltype(auto) x2d(i);  // decltype(x2d) is int
auto      x3a = i;      // decltype(x3a) is int
decltype(auto) x3d = i;  // decltype(x3d) is int
auto      x4a = (i);     // decltype(x4a) is int
decltype(auto) x4d = (i); // decltype(x4d) is int&
auto      x5a = f();     // decltype(x5a) is int
decltype(auto) x5d = f(); // decltype(x5d) is int&&
auto      x6a = { 1, 2 }; // decltype(x6a) is std::initializer_list<int>
decltype(auto) x6d = { 1, 2 }; // error, { 1, 2 } is not an expression
auto      *x7a = &i;     // decltype(x7a) is int*
decltype(auto)*x7d = &i;  // error, declared type is not plain decltype(auto)
```

—end example]

Modify §14.1 [temp.param] paragraph 5:

A non-type *template-parameter* shall have one of the following (optionally *cv-qualified*) types:

- integral or enumeration type,
- pointer to object or pointer to function,
- lvalue reference to object or lvalue reference to function,
- pointer to member,
- `std::nullptr_t`,
- a type that contains a placeholder type designated by `auto` or `decltype(auto)` (7.1.6.4).

Modify §14.4.2 [temp.arg.nontype] paragraph 1:

A *template-argument* for a non-type *template-parameter* shall be a converted constant expression (5.20) of the type of the *template-parameter*. If the type of the *template-parameter* contains a placeholder type (7.1.6.4, 14.1), the deduced parameter type is determined from the type of the *template-argument* using the rules for template argument deduction from a function call (14.9.2.1). If a deduced parameter type is not permitted for a *template-parameter* declaration (14.1), the program is ill-formed. For a non-type *template-parameter* of reference or pointer type, the value of the constant expression shall not refer to (or for a pointer type, shall not be the address of):

- a subobject (1.8),

- a temporary object (12.2),
- a string literal (2.13.5),
- the result of a typeid expression (5.2.8), or
- a predefined __func__ variable (8.4.1).

[*Note*: If the *template-argument* represents a set of overloaded functions (or a pointer or member pointer to such), the matching function is selected from the set (13.4). —*end note*]

Modify §14.4.2 [temp.arg.nontype] paragraph 2:

[*Example*:

```
template<const int* pci> struct X { /* ... */ };
int ai[10];
X<ai> xi; // array to pointer and qualification conversions

struct Y { /* ... */ };
template<const Y& b> struct Z { /* ... */ };
Y y;
Z<y> z; // no conversion, but note extra cv-qualification

template<int (&pa)[5]> struct W { /* ... */ };
int b[5];
W<b> w; // no conversion

void f(char);
void f(int);

template<void (*pf)(int)> struct A { /* ... */ };
A<&f> a; // selects f(int)

template<auto n> struct B { /* ... */ };
B<5> b1; // OK: template parameter type is int
B<'a'> b2; // OK: template parameter type is char
B<2.5> b3; // error: template parameter type cannot be double
```

—*end example*]

Modify §14.6.5 [temp.class.spec] paragraph 9 list item 9.2:

The type of a template parameter corresponding to a specialized non-type argument shall not be dependent on a parameter of the specialization that appears as a *type-specifier* in the *type-specifier-seq* of an argument in the argument list of the specialization. Otherwise, the corresponding arguments are deduced from the type of the non-type argument. [*Example*:

```
template <class T, T t> struct C {};
template <class T> struct C<T, 1>; // error
template< int X, int (*array_ptr)[X] > class A {};
int array[5];
template< int X > class A<X,&array> { }; // error
template <auto f> class B {};
template <class R, class... Args, R (* f)(Args...)> class B<f> {} // ok
```

—*end example*]

Modify §14.7.2.2 [temp.dep.expr] paragraph 3:

An *id-expression* is type-dependent if it contains

- an *identifier* associated by name lookup with one or more declarations declared with a dependent type,
- an *identifier* associated by name lookup with a non-type *template-parameter* declared with a type that contains a placeholder type (7.1.6.4).
- an *identifier* associated by name lookup with one or more declarations of member functions of the current instantiation declared with a return type that contains a placeholder type ~~(7.1.6.4)~~,
- the *identifier* __func__ (8.4.1), where any enclosing function is a template, a member of a class template, or a generic lambda,
- a *template-id* that is dependent,

- a *conversion-function-id* that specifies a dependent type, or
- a *nested-name-specifier* or a *qualified-id* that names a member of an unknown specialization;

or if it names a dependent member of the current instantiation that is a static data member of type “array of unknown bound of T” for some T (14.6.1.3). Expressions of the following forms are type-dependent only if the type specified by the *type-id*, *simple-type-specifier* or *new-type-id* is dependent, even if any subexpression is type-dependent:

```

simple-type-specifier ( expression-listopt )
::opt new new-placementopt new-type-id new-initializeropt
::opt new new-placementopt ( type-id ) new-initializeropt
dynamic_cast < type-id > ( expression )
static_cast < type-id > ( expression )
const_cast < type-id > ( expression )
reinterpret_cast < type-id > ( expression )
( type-id ) cast-expression

```

Modify §14.9.2.5 [temp.deduct.type] paragraph 5:

The non-deduced contexts are:

- The *nested-name-specifier* of a type that was specified using a *qualified-id*.
- The *expression* of a *decltype-specifier*.
- A non-type template argument or an array bound in which a subexpression references a template parameter.
[*Note: Deduction is still performed based on the type of a non-type template argument. —end note*]
- A template parameter used in the parameter type of a function parameter that has a default argument that is being used in the call for which argument deduction is being done.
- A function parameter for which argument deduction cannot be done because the associated function argument is a function, or a set of overloaded functions (13.4), and one or more of the following apply:
 - more than one function matches the function parameter type (resulting in an ambiguous deduction), or
 - no function matches the function parameter type, or
 - the set of functions supplied as an argument contains one or more function templates.
- A function parameter for which the associated argument is an initializer list (8.5.4) but the parameter does not have a type for which deduction from an initializer list is specified (14.9.2.1). [*Example:*

```

template<class T> void g(T);
g({1,2,3}); // error: no argument deduced for T

```

—*end example*]

- A function parameter pack that does not occur at the end of the *parameter-declaration-list*.

Acknowledgments

Numerous people gave constructive feedback regarding the use of `auto` in template parameter lists in an [isocpp.org](https://ericniebler.com/2015/07/20/constexpr-variables/) discussion [thread](#).

Special thanks to Mike Spertus and Gabriel dos Reis for their invaluable analysis and assistance.

Thanks also to Andrew Sutton for assistance and instruction in identifying potential areas of conflict with the Concepts TS.